CSIS 402: Oral Defense & Evaluation Guide

Strict Professor & Examiner Assessment: Anticipated Viva Questions, Exact Model Answers, and Grading Rubric

EXAMINER MEMORANDUM TO STUDENTS:
"Do not expect a superficial code review. In this oral evaluation, I will test whether you truly understand every single byte, flag, register, and cycle in your 8086 assembly implementation. If you memorized code or copied snippets without grasping the underlying hardware architecture and Galois Field mathematics, you will receive zero marks on the spot. Study each question below with utmost rigor."

CATEGORY I: Hardware Architecture, Memory & Registers

QUESTION 1: Why is the S-Box 256 bytes when the AES block is only 128 bits?

Examiner Persona: *"You declared a 256-byte table for sbox in your DATA segment, but the project specifications explicitly state this is a 128-bit AES implementation! Did you mistakenly implement AES-256? Explain this discrepancy immediately!"*

Model Answer to Deliver:
No Professor, AES-128 operates on a 128-bit data block (which is exactly 16 bytes). That 16-byte block is stored in our state variable. However, the SubBytes step replaces each individual byte independently. An 8-bit byte can take on 2^8 = 256 possible values, ranging from 00h to FFh. Therefore, to provide a substitute for any possible byte that might appear in the state, the lookup dictionary must contain all 256 possible mappings (16 rows x 16 columns = 256 bytes). The 128 bits refers to the data block size, while 256 bytes is the size of the substitution table.

[+] What earns full marks: Clear distinction between block/state size (128 bits = 16 bytes) and byte domain size (2^8 = 256 possible byte values).
[-] Fatal mistake (Zero points): Claiming that 256 bytes means AES-256 or confusing bits with bytes.

QUESTION 2: Mandatory Data Segment Initialization (MOV AX, @DATA / MOV DS, AX)

Examiner Persona: *"Look at the beginning of your MAIN procedure. Why did you write MOV AX, @DATA followed by MOV DS, AX? Why didn't you just write MOV DS, @DATA directly and save an instruction?"*

Model Answer to Deliver:
The 8086 microprocessor architecture does not support moving an immediate value directly into a segment register (such as DS, CS, SS, or ES). The opcode format for MOV with a segment register requires the source operand to be a general-purpose register or a 16-bit memory location. Therefore, we must load the immediate base address @DATA into a general-purpose register like AX first, and then transfer AX into DS. Furthermore, while the DOS loader automatically sets CS and IP upon launching the executable, it leaves DS pointing to the Program Segment Prefix (PSP), so the program must manually initialize DS to access variables in the DATA segment.

[+] What earns full marks: Citing 8086 hardware opcode constraints: immediate-to-segment register moves are illegal in 8086.
[-] Fatal mistake (Zero points): Saying "it's just standard MASM syntax" without knowing the CPU instruction limitation.

QUESTION 3: Addressing Modes: S-Box Lookup & Memory Displacements

Examiner Persona: *"In SubBytes, you wrote LEA BX, sbox; ADD BX, AX; MOV AL, [BX]. Exactly what addressing mode is [BX]? Could you have used [DX] or [AX] instead? What segment register does [BX] pair with?"*

Model Answer to Deliver:
The instruction MOV AL, [BX] uses Register Indirect Addressing. In the 16-bit 8086 architecture, only four specific registers can serve as base or index registers inside brackets for memory addressing: BX, BP, SI, and DI. Using [AX], [CX], or [DX] as a memory pointer is physically unsupported by the 8086 hardware and causes a compile-time syntax error. By default, when using BX, SI, or DI to reference memory, the processor pairs them with the Data Segment register (DS) to compute the 20-bit physical address: Physical Address = (DS * 16) + BX. If BP were used, it would pair with SS (Stack Segment) by default.

[+] What earns full marks: Naming Register Indirect Addressing, stating the 4 valid registers (BX, BP, SI, DI), and noting DS vs SS segment defaults.
[-] Fatal mistake (Zero points): Claiming DX or AX can be used inside memory brackets [DX].

QUESTION 4: Preserving the CX Counter across Nested MixColumns Loops

Examiner Persona: *"In MixColumns, you wrote PUSH CX before the column math and POP CX after it. Why? Does your xtime procedure touch CX? If not, why waste stack cycles on PUSH and POP?"*

Model Answer to Deliver:
MixColumns loops through all 4 columns using the 8086 LOOP instruction, which automatically decrements CX and branches until CX is zero. While our xtime procedure currently only uses AL and the flags, using PUSH CX and POP CX is defensive systems programming: inside the column loop, extensive calculations are carried out, and should any helper or macro expansion modify CX or CL (e.g. for variable bit shifts like SHL reg, CL), the outer column counter would be corrupted, causing an infinite loop or premature termination. Preserving CX on the stack guarantees loop integrity.

[+] What earns full marks: Explaining that LOOP is tied to CX, and stack preservation shields the loop counter from nested clobbering.
[-] Fatal mistake (Zero points): Not knowing that LOOP inherently modifies CX.

CATEGORY II: Cryptographic & Galois Field Mathematics

QUESTION 5: Galois Field GF(2^8) Multiplication and the xtime Procedure

Examiner Persona: *"Explain the mathematical logic of xtime. You do SHL AL, 1, then JNC xtime_done, then XOR AL, 1Bh. Why 1Bh and not 11Bh? Which flag is checked by JNC?"*

Model Answer to Deliver:
In AES, field elements are polynomials in GF(2^8) modulo the irreducible polynomial m(x) = x^8 + x^4 + x^3 + x + 1, which in binary is 100011011b (11Bh). Multiplying a polynomial by x corresponds to shifting left by 1 bit (SHL AL, 1). Bit 7 (the coefficient of x^7) is shifted into the Carry Flag (CF). If CF = 0, the result degree is < 8, so no reduction is necessary (JNC checks CF = 0). If CF = 1, degree 8 is reached, meaning we must reduce by subtracting m(x). In GF(2), subtraction is XOR. Because bit 8 is already shifted out of the byte into CF, we only need to XOR the remaining 8 bits with the lower 8 bits of m(x), which is 00011011b = 1Bh!

[+] What earns full marks: Explaining polynomial representation, SHL as multiplying by x, CF receiving bit 7, and why the 9th bit (100h) is discarded so 11Bh becomes 1Bh.
[-] Fatal mistake (Zero points): Thinking 1Bh is an arbitrary "magic number" without knowing the AES irreducible polynomial.

QUESTION 6: ShiftRows: Row 3 Circular Rotation Equivalence

Examiner Persona: *"In ShiftRows, you shifted Row 3 by rotating right by 1 byte. But FIPS-197 states that Row 3 is shifted cyclically LEFT by 3 bytes! Did you fail to follow the AES specification?"*

Model Answer to Deliver:
No Professor, they are mathematically identical. In a circular array of 4 elements (length N = 4), shifting left by K positions is completely equivalent to shifting right by (N - K) positions: (4 - 3) = 1 position right. Rotating left by 3 elements ([3], [7], [11], [15]) puts element [15] into [3], [3] into [7], [7] into [11], and [11] into [15]. A circular right shift by 1 position produces the exact same permutation with fewer temporary register moves and fewer clock cycles on the 8086.

[+] What earns full marks: Demonstrating modular arithmetic proof: Left Shift by 3 is congruent to Right Shift by (4 - 3) = 1 in a 4-element cycle.
[-] Fatal mistake (Zero points): Admitting defeat or claiming it was an error.

QUESTION 7: Why MixColumns Requires a Temporary Buffer (col_temp)

Examiner Persona: *"Why did you allocate col_temp DB 4 DUP(0)? Why didn't you write the new bytes directly back into the state array as you computed them?"*

Model Answer to Deliver:
In MixColumns, each of the four output bytes (r0, r1, r2, r3) is a linear combination of all four original column bytes (s0, s1, s2, s3). If we overwrote state[SI+0] with r0 immediately, then when computing r1 = s0 XOR 2*s1 XOR 3*s2 XOR s3, the instruction MOV BL, [SI+0] would read the newly computed r0 instead of the original s0, causing severe data corruption for r1, r2, and r3. Therefore, all 4 results must be calculated from the pristine inputs and buffered in col_temp before being copied back into state.

[+] What earns full marks: Identifying read-after-write data dependency / in-place clobbering across the linear system of equations.
[-] Fatal mistake (Zero points): Thinking the buffer is just for "debugging".

QUESTION 8: AddRoundKey with Key = 16 bytes of 0FFh

Examiner Persona: *"Milestone 1 uses a fixed round key of FFFFFFFFFFFFFFFF... Why didn't you just use the NOT instruction on the state bytes instead of setting up a key array, pointers, and XOR?"*

Model Answer to Deliver:
Mathematically, XORing any bit with 1 flips the bit (A XOR 1 = NOT A), so XORing with 0FFh is functionally identical to the NOT instruction. However, coding it as NOT [SI] would violate software engineering principles and the milestone requirements: this project requires a general AddRoundKey module for Milestone 1 that can be reused in Milestone 2. In Milestone 2, round keys will be generated via the AES Key Expansion schedule and will contain arbitrary pseudorandom bytes. Our macro uses general source/destination pointers (SI and DI) with XOR AL, AH, so it works out of the box for any round key.

[+] What earns full marks: Explaining the truth table of XOR with 1 as inversion, while defending code generality for Milestone 2 key expansion.
[-] Fatal mistake (Zero points): Not realizing that XOR with 0FFh is the NOT operation.

CATEGORY III: Macros, Assembler Directives & Software Design

QUESTION 9: Why MACROs vs PROCEDURES? And the purpose of LOCAL

Examiner Persona: *"Why are SubBytes and MixColumns implemented as MACROs, while xtime is a PROC? What does the LOCAL directive do inside your macros?"*

Model Answer to Deliver:
Milestone 1 explicitly mandated using MACROs for the four round operations. A macro is expanded inline by the assembler at preprocessing time wherever invoked—there is no CALL or RET instruction, meaning zero stack overhead and zero branch delay, but with increased binary size if invoked repeatedly. In contrast, xtime is called 16 times per round (4 times per column x 4 columns); defining xtime as a PROCEDURE avoids duplicating that logic 16 times in memory, keeping code compact. Furthermore, because a macro is expanded inline, any label defined inside it (like sb_loop) would be duplicated if the macro is invoked more than once, causing duplicate symbol errors. The LOCAL directive tells MASM to generate unique internal labels (e.g. ??0001) for each expansion.

[+] What earns full marks: Contrasting inline expansion vs CALL/RET stack overhead, and accurately explaining why LOCAL prevents duplicate label collisions.
[-] Fatal mistake (Zero points): Not knowing the difference between a macro and a procedure in assembly.

QUESTION 10: Column-Major Ordering and Endianness in 8086

Examiner Persona: *"How is the 4x4 state matrix organized in memory? Does Intel's Little-Endian architecture affect byte access in your AES implementation?"*

Model Answer to Deliver:
AES organizes the state matrix in COLUMN-MAJOR order: bytes 0..3 constitute Column 0, bytes 4..7 Column 1, bytes 8..11 Column 2, and bytes 12..15 Column 3. Regarding endianness, Intel 8086 is Little-Endian when loading multibyte words (low byte at low address). However, in our implementation, all AES operations—SubBytes, ShiftRows, MixColumns, and AddRoundKey—access memory strictly as single individual BYTES (e.g., MOV AL, [SI]). Because endianness only dictates how multibyte words or doublewords are arranged in memory, single-byte accesses are completely immune to endianness variations.

[+] What earns full marks: Stating column-major mapping and clarifying that Little-Endianness only applies to multi-byte words, not byte-by-byte reads.
[-] Fatal mistake (Zero points): Stating AES is row-major, or claiming Little-Endian reversed their bytes.